Name: Shrusti Dnyandev Sontakke

PRN No:202501120039

Division:DS-1

Batch:D-12

Codetantra Practical 1

1.1.1 Problem Statement

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula:

where:

is the mass of the object (in kilograms).

is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
mass=float(input())
velocity=float(input())
momentum=mass*velocity
print(f"{momentum:.2f}kgm/s")
```

Output:

Average time	Maximum time
0.008 s	**0.010 s**
8.00 ms	10.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

10 ms

Expected output	Actual output
5.0	5.0
10.0	10.0
50.00kgm/s	50.00kgm/s

Test case 2

10 ms

Expected output	Actual output
2.3	2.3
4.8	4.8
11.04kgm/s	11.04kgm/s

1.1.2 problem statement

Write a Python program that accepts an integer as input. Depending on the number of digits in .

Constraints:

1 ≤ ≤ 999

Input Format:

- The input consists of a single integer .

Output Format:

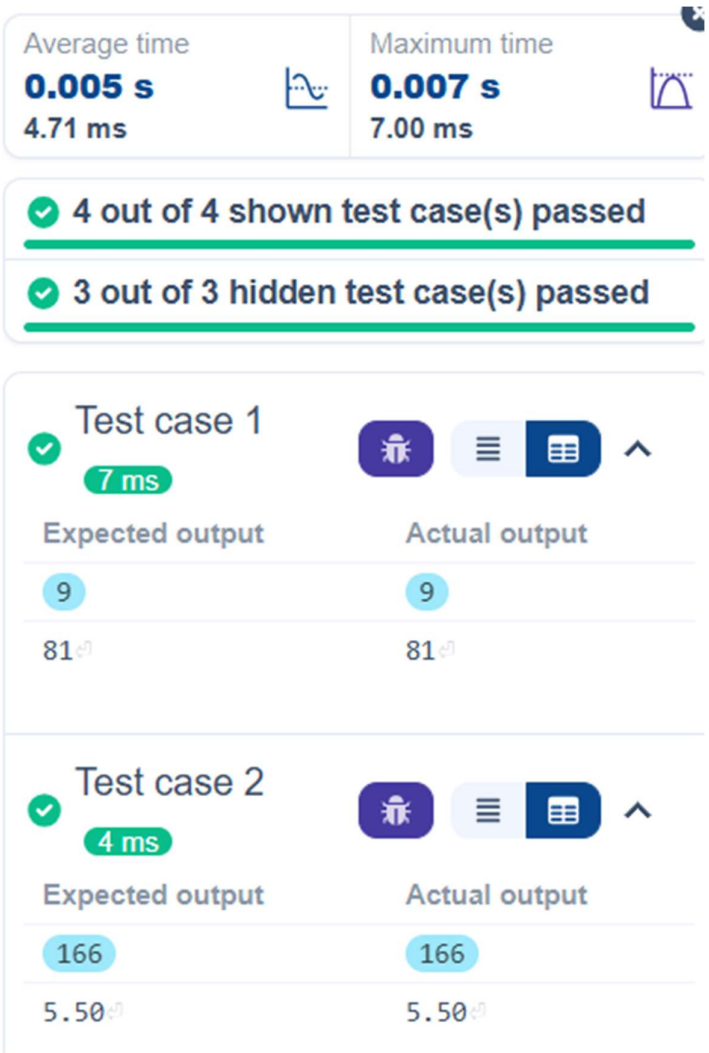
- If is a single-digit number, print its square.
- If is a two-digit number, print its square root (rounded to two decimal places).
- If is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Code:

```
import math
number=int(input())
if 1<= number <=9:
    print(number ** 2)
elif 10 <= number <=99:
    print(f"{math.sqrt(number):.2f}")
elif 100 <= number <=999:
    print(f"{math.cbrt(number):.2f}")
```

```
else:
    print("Invalid")
```

Output:



Test case 3

4 ms

Expected output	Actual output
24	24
4.90	4.90

Test case 4

5 ms

Expected output	Actual output
3456	3456
Invalid	Invalid

1.1.3 problem statement

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format .
- The second line contains the second date in the format .

Output Format:

- An integer representing the number of days between the given dates.

Note:

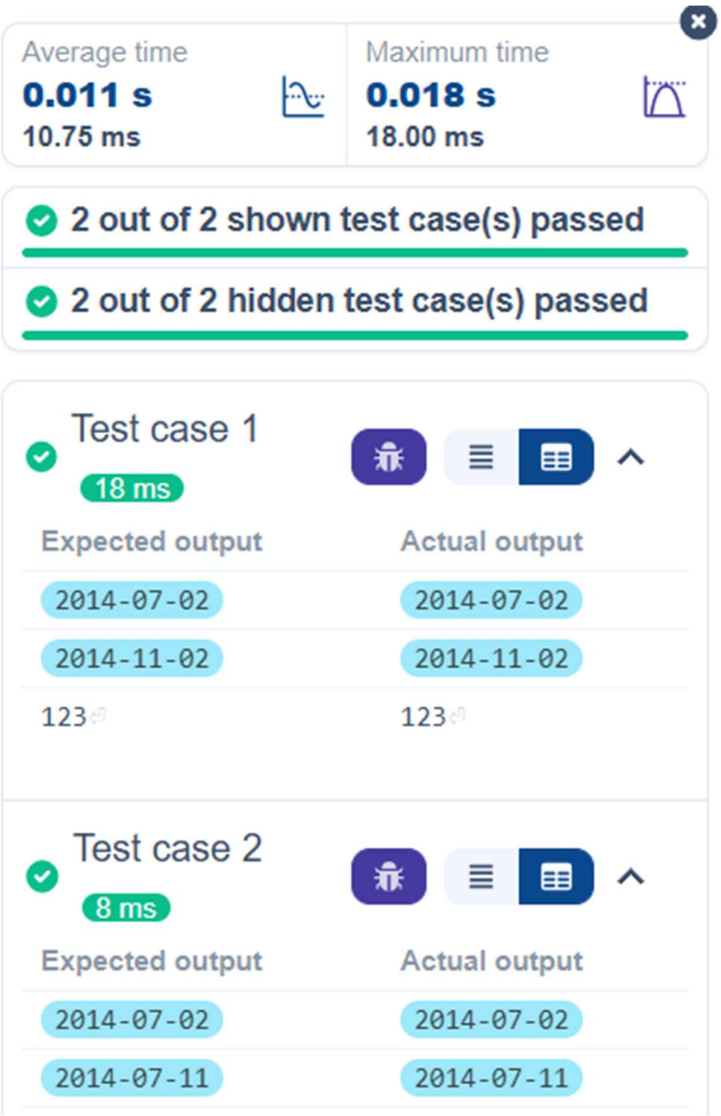
- The first date should always be considered earlier than the second date.

Code:

```
from datetime import date

from datetime import datetime

#read two dates as input
date1=input().strip()
date2=input().strip()

# convert string to datetime object
d1=datetime.strptime(date1, "%Y-%m-%d")
d2=datetime.strptime(date2, "%Y-%m-%d")

#calculate difference
difference=d2-d1
```

```
#print number of days
print(difference.days)
```

Output:



1.1.4 problem statement

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

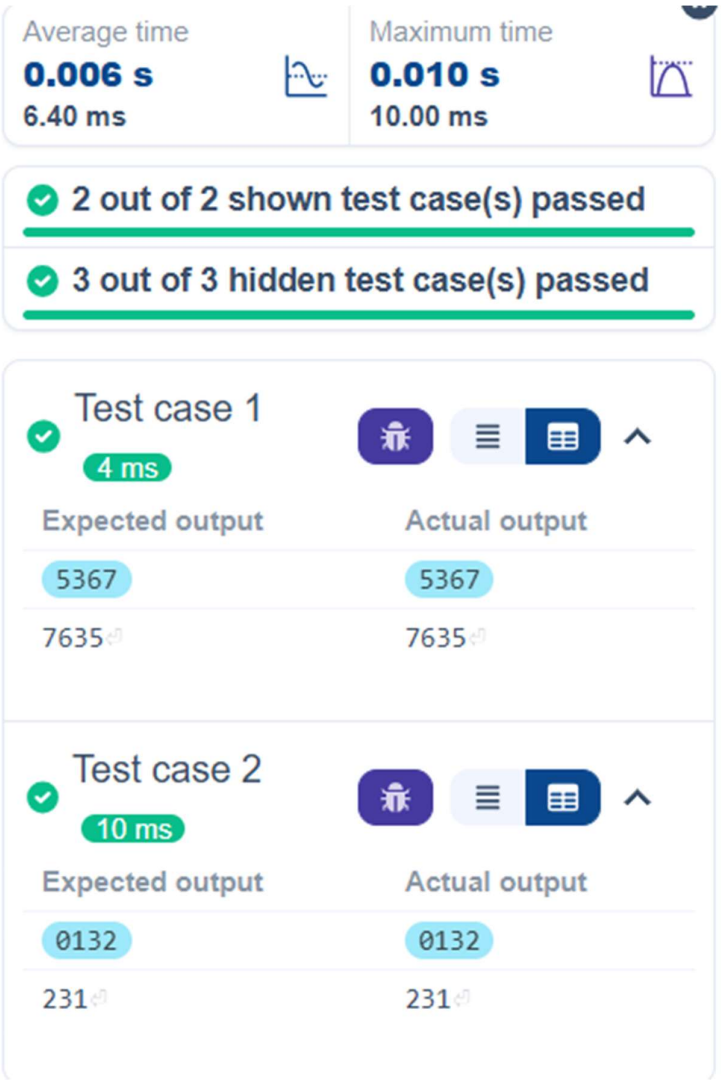
Output Format:

- The output prints a single integer, which is the reversed number.

Code:

```
num=int(input())   #read the integer input

#reverse the digits of the number
reversed_num=int(str(num)[::-1])

#print the reversed number
print(reversed_num)
```

Output:



1.1.5 problem statement

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

  <**number**> x <**multiplier**> = <**result**>

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Code:

```
num=int(input())
for i in range(1,11):
    print(f"{num} x {i} = {num * i}")
```

Output:

Test case 2

4 ms

Expected output	Actual output
5	5
5 x 1 = 5	5 x 1 = 5
5 x 2 = 10	5 x 2 = 10
5 x 3 = 15	5 x 3 = 15
5 x 4 = 20	5 x 4 = 20
5 x 5 = 25	5 x 5 = 25
5 x 6 = 30	5 x 6 = 30
5 x 7 = 35	5 x 7 = 35
5 x 8 = 40	5 x 8 = 40
5 x 9 = 45	5 x 9 = 45
5 x 10 = 50	5 x 10 = 50

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

11 ms

Expected output	Actual output
8	8
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40
8 x 6 = 48	8 x 6 = 48
8 x 7 = 56	8 x 7 = 56
8 x 8 = 64	8 x 8 = 64
8 x 9 = 72	8 x 9 = 72
8 x 10 = 80	8 x 10 = 80

Lab Assignment:

1.2.1 problem statement

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer , the number of courses.

- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Code:

```
def cal(marks,total_courses):
    if any(mark<40 for mark in marks):
        return "Fail"
```

```
    total_marks=sum(marks)
    aggregate_percentage=(total_marks/(total_cour
ses *100))*100

    if aggregate_percentage>75:
        grade="Distinction"
    elif aggregate_percentage >= 60:
        grade="First Division"
    elif aggregate_percentage >= 50:
        grade="Second Division"
    elif aggregate_percentage >= 40:
        grade="Third Division"
    return (aggregate_percentage,grade)

num_courses=int(input())
marks=list(map(int,input().split()))
result= cal(marks,num_courses)
if result == "Fail":
    print("Fail")
else:
```

```
    aggregate_percentage,grade=result
    print(f"Aggregate Percentage: {aggregate_percentage:.2f}")
    print(f"Grade: {grade}")
```

Output:

Average time	Maximum time
0.009 s	**0.012 s**
8.50 ms	12.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 1

12 ms

Expected output	Actual output
4	4
55 65 78 89	55 65 78 89
Aggregate Percentage: 71.75	Aggregate Percentage: 71.75
Grade: First Division	Grade: First Division

Test case 2

10 ms

Expected output	Actual output
5	5
56 78 97 86 93	56 78 97 86 93
Aggregate Percentage: 82.00	Aggregate Percentage: 82.00
Grade: Distinction	Grade: Distinction

Test case 3

8 ms

Expected output	Actual output
5	5
99 85 43 97 34	99 85 43 97 34
Fail	Fail

Test case 4

6 ms

Expected output	Actual output
3	3
55 54 53	55 54 53
Aggregate Percentage: 54.00	Aggregate Percentage: 54.00
Grade: Second Division	Grade: Second Division

Test case 5

10 ms

Expected output	Actual output
5	5
40 45 42 48 41	40 45 42 48 41
Aggregate Percentage: 43.20	Aggregate Percentage: 43.20
Grade: Third Division	Grade: Third Division

1.2.2 problem statement

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

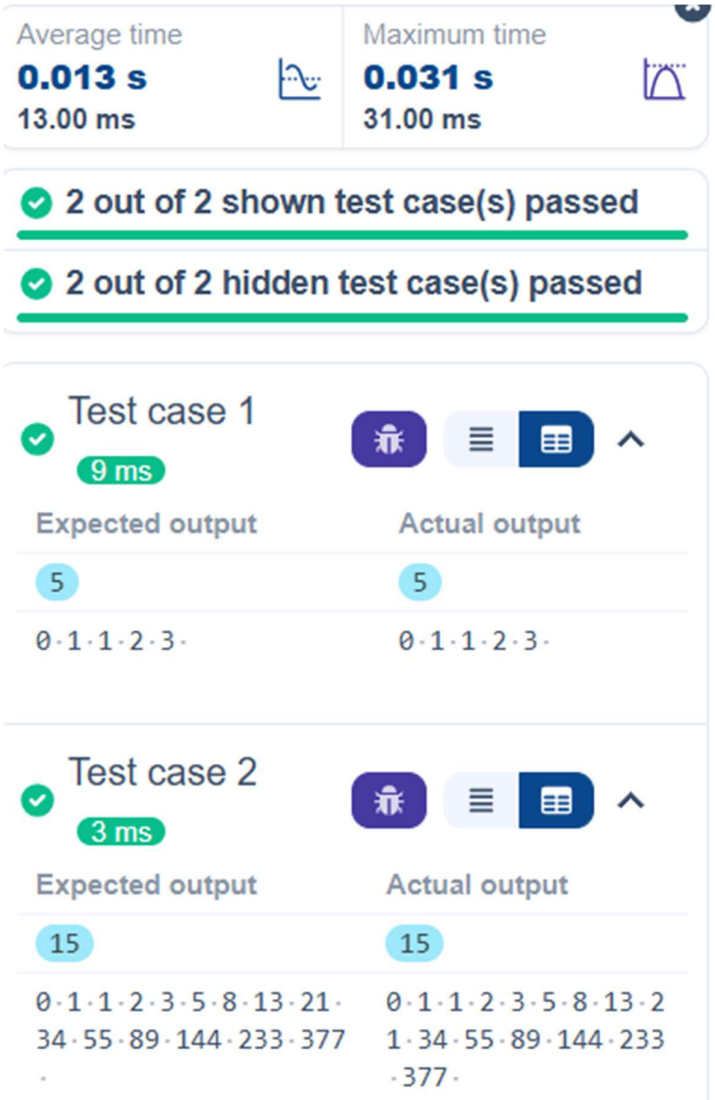
Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

Code:

```
def fibonacci(n):


    if n==1:
        return 0
    elif n==2:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)
n = int(input())
for i in range(1, n + 1):
    print(fibonacci(i), end=" ")
```

Output:



1.2.3 problem statement

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Code:

```
n=int(input())

#generate right-angled triangle pattern

for i in range(1, n+1):
    print('* ' * i)
```

Output:

1.2.4 problem statement

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Code:

```
n=int(input())
for i in range(1,n+1):
    for j in range(1, i+1):
        print(j, end=" ")
    print()
```

Output:

Average time	Maximum time
0.005 s	**0.007 s**
5.25 ms	7.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

7 ms

Expected output	Actual output
5	5
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5

Test case 2

4 ms

Expected output	Actual output
8	8
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5
1 2 3 4 5 6	1 2 3 4 5 6
1 2 3 4 5 6 7	1 2 3 4 5 6 7
1 2 3 4 5 6 7 8	1 2 3 4 5 6 7 8